WikiPulse: Real-Time Detection and Analysis of Wikipedia Knowledge Shifts

Authors: Kshama Jain, Senthil Raj K, Pothukanuri Sai Venkat, Akash H S

Course: DA231o - Data Engineering at Scale, IISc, 2026

Problem Definition

Wikipedia is an evolving knowledge base where real-world events trigger acute shifts in editing behaviour, including velocity spikes, rapid revisions, and new contributor arrivals. Because these signals are currently fragmented across live streams, historical metrics, and complex page relationships, distinguishing meaningful knowledge shifts from routine editing is difficult.

WikiPulse will deliver a distributed data platform that detects anomalous editing activity in real time, benchmarks it against historical baselines, and contextualizes how these updates spread across related topics and locations.

Motivation

Fixed rules like "15 edits in five minutes" fail because baseline behaviour varies. Fifteen edits is exceptional for quiet pages but routine for active ones. A useful system must combine live activity with page-specific history and evidence.

The project fits Data Engineering at Scale by integrating continuous ingestion, large historical datasets, distributed aggregation, machine learning, graph relationships, and repeatable scale experiments. The final deliverables will feature live, historical, graph/map, and engineering dashboards.

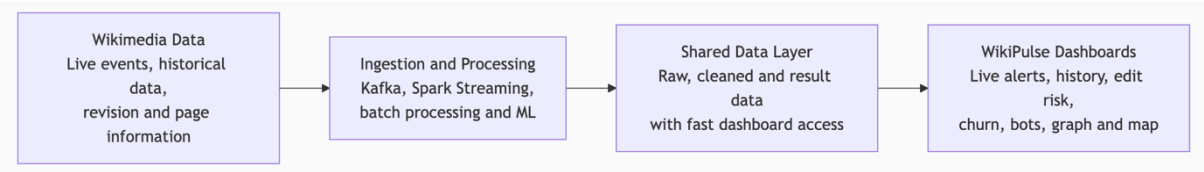
Design goals

Durable and replay able ingestion; event-time processing that tolerates late and out-of-order events; fault-tolerant, checkpointed storage; horizontal scaling across partitions and workers; explainable alerts; reproducible experiments.

Supported Features

- **WikiPulse Knowledge-Shift Intelligence:** Detect unusual page activity and explain the score using edits, editors, content movement, history, and related pages.
- **Breaking News Radar:** Identify sudden activity bursts and use delayed pageviews as supporting evidence; an alert is not called confirmed news without additional evidence.
- **Edit-Risk and Reversion Monitor:** Score selected revisions using simple checks and Wikimedia Revert Risk, then record whether they are later reverted.
- **Global Geospatial Knowledge Map:** Plot the location of active article subjects using Wikipedia coordinates and Wikidata; editor IP addresses will not be used.
- **Knowledge Churn and Bot Ratio:** Measure page growth, rewriting, edit speed, contributor diversity, and the human-versus-bot split.

High-level design



Data sources

Wikimedia Event Streams; Wikimedia Analytics API for edits, pageviews, and byte changes; MediaWiki History dumps; MediaWiki revision comparison, links, backlinks, coordinates, and page properties; Wikidata; and Lift Wing Revert Risk.

Data Models

Raw edit, Page windows, Historical baseline, Alerts, Graph edge.

Evaluation Approach

Experiment plan:

- Validate Spark calculations against manually checked event samples and replay the same data.
- Compare a fixed threshold, historical page score, and Spark ML anomaly score.
- Compare one useful aggregation using RDDs and Data Frames.
- Change partitions, workers, and input size to measure shuffle, strong scaling, and weak scaling.
- Stop and restart processing to measure backlog recovery and duplicate handling.
- Compare JSON and Parquet size and dashboard query time.

Performance metrics for success:

- At least 99% of valid accepted events reach raw storage.
- No duplicate alerts are produced for the same event/window after restart.
- A closed five-minute window produces its result within one minute under the target load.
- Processing rate remains at or above input rate during the normal run, and the Kafka backlog returns to zero after a controlled restart.
- Runtime, shuffle, memory, and storage size are reported for every scale experiment.
- Dashboard queries meet an initial target of under two seconds for stored result tables.

Feature metrics for success:

- All injected knowledge-shift patterns are detected and explained with at least two reasons.
- The historical score creates fewer obvious false alerts than the fixed threshold in a reviewed sample.
- At least 90% of selected revision comparisons return usable change data.
- At least 95% of supported Revert Risk requests return a usable score during testing; the system continues if the API is unavailable.
- Risk thresholds are compared against later revert outcomes rather than assuming that 0.80 is always correct.
- Every map point stores its location source and quality; pages without locations remain valid alerts.
- Live churn totals match manually calculated sample totals.
- Historical charts show the source's last available date and distinguish missing data

Timeline and Milestones (8 weeks)

Week	Planned Work	Expected Milestone
1	Confirm APIs, event schema, and dashboard specs.	Feasibility and shared data model complete.
2	Build Kafka ingestion, raw storage, and replay tool.	Events live in Kafka and Parquet.
3	Build Spark streaming windows and live metrics.	First live dashboard operational.
4	Build historical ingestion, baselines, and rankings.	Historical comparison enabled.
5	Implement anomaly scoring and Spark ML tests.	Repeatable alerts detected and evaluated.
6	Add contextual features (graph, map, Revert Risk).	End-to-end investigation flow ready.
7	Execute scaling, load, and failure-recovery tests.	Engineering scale results finalized.
8	Finalize integration, report, and backup demo.	Reproducible system and presentation ready.